

# 第三天 迴圈

大里 APCS 營隊 · 讓電腦重複幫你做事

邱皇愷

## 今天的目標

- 學會三種迴圈：**for** / **while** / **do-while**
- 掌握迴圈三元素：初始值、條件、更新
- 用巢狀迴圈畫出乘法表與各種圖形
- 善用 **break** / **continue** 控制迴圈流程
- 每教完一段，馬上到 itouOJ 練一題，完成 Day3 的 6 題

開場暖身

## 回顧昨天

- `if / else if / else`` 讓程式**做一次決定**
- 條件要由**嚴格到寬鬆**排列，否則會被提早攔截
- `&&``、`||``、`!`` 可以把多個條件組合起來
- 今天要學的，是讓程式**一直做決定、一直重複**

## 暖身問題（不用寫程式）

如果要在紙上，把 1 加到 100，你會怎麼做？寫 100 個加號嗎？

有沒有更輕鬆的做法？

想 30 秒，等一下公佈參考答案。

## 暖身問題：參考答案

- 高斯的做法：頭尾配對，`1+100`、`2+99` .....每對都是 101，共 50 對， $101 \times 50 = 5050$
- 但如果數字沒有規律呢？例如使用者臨時輸入一串亂數：`12、45、3、78、9.....`？
- 這時候完全沒有「頭尾配對」可言，最直接的做法：**準備一個小計，從第一個數字開始，一個一個加上去**
- 這個「一個一個做」的動作，就是今天要學的**迴圈**

高斯的技巧很聰明，但只適用於「連續整數求和」這種特殊情況。迴圈的威力在於：不管數字有沒有規律，它都能算，而且完全不用你自己想公式。

為什麼需要迴圈？

# 沒有迴圈的世界

如果要印出 1 到 5：

```
cout << 1 << endl;  
cout << 2 << endl;  
cout << 3 << endl;  
cout << 4 << endl;  
cout << 5 << endl;
```

那如果要印 1 到 10000 呢？🤖

只要是「重複做同樣的事」，就交給迴圈。



## 生活中的迴圈

- 刷牙：「刷一下」重複 20 次
- 發考卷：「發給一個人」重複「班上人數」次
- 掃地：「掃一下」重複到「地板乾淨」為止

注意最後一個例子跟前兩個不一樣：前兩個知道**確切次數**，最後一個只知道**什麼時候該停**，不知道會重複幾次。這對應到等一下要學的兩種不同的迴圈。

# 沒有迴圈，程式碼會怎樣爆炸

想印出 1 到 100 的每個數字：

## 不用迴圈

100 行幾乎一樣的 `cout`，想印到 1000 就要重寫成 1000 行

## 用迴圈

3 行程式碼，想印到 1000 只要把條件的 100 改成 1000

程式越長，出錯的機會越多。迴圈不只是「少打字」，更是少一個出錯的地方。

for 迴圈

## for 迴圈三元素

```
for (int i = 1; i <= 5; i++) {  
    cout << i << " ";    // 1 2 3 4 5  
}
```

`int i = 1`

**初始值**

從哪裡開始  
整個迴圈只做一次

`i <= 5`

**條件**

還要繼續嗎  
每一輪開頭檢查

`i++`

**更新**

下一輪怎麼變  
每一輪結束時做

## 拆開看①：初始值

```
for (int i = 1; i <= 5; i++) {  
    cout << i << " ";  
}
```

`int i = 1` 只會在**進入迴圈前**執行一次。它同時做了兩件事：宣告一個新變數 `i`，並給它一個起始值。

## 拆開看②：條件

```
for (int i = 1; i <= 5; i++) {  
    cout << i << " ";  
}
```

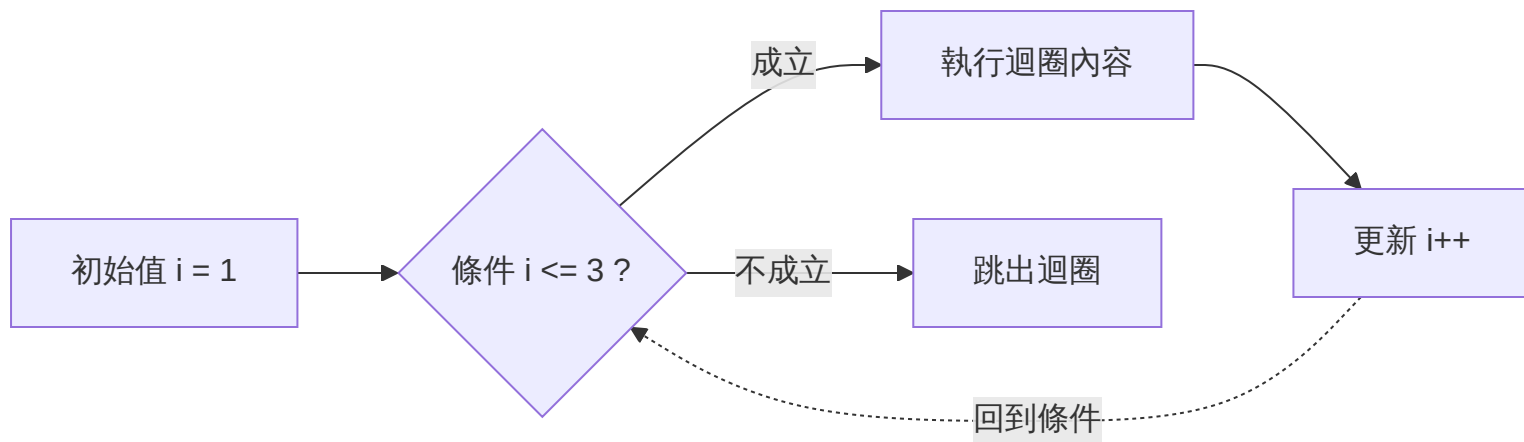
``i <= 5`` 在每一輪開始前都會檢查一次。成立才會執行大括號裡的內容，不成立就直接跳出迴圈，一步都不會多做。

## 拆開看③：更新

```
for (int i = 1; i <= 5; i++) {  
    cout << i << " ";  
}
```

``i++`` 在**每一輪結束後**才執行，然後才回頭再檢查一次條件。順序是：檢查條件 → 執行內容 → 更新 → 再檢查條件.....一直循環。

## for 迴圈的執行順序



初始值只做一次；條件每輪開頭檢查；更新在每輪結束時做。



## 卡住的時候，就手動跑一遍

```
for (int i = 1; i <= 3; i++) {  
    cout << i;  
}
```

| 輪次    | 檢查 <code>i &lt;= 3</code> | 輸出             | 更新後 i |
|-------|---------------------------|----------------|-------|
| 第 1 輪 | $1 \leq 3$ ✓              | <code>1</code> | 2     |
| 第 2 輪 | $2 \leq 3$ ✓              | <code>2</code> | 3     |
| 第 3 輪 | $3 \leq 3$ ✓              | <code>3</code> | 4     |
| 第 4 輪 | $4 \leq 3$ ✗              | —              | 結束    |

這張表比盯著程式碼看有用一百倍。迴圈寫不出來就先畫它。

## 練習：手動跑一遍

```
for (int i = 0; i < 4; i++) {  
    cout << i * 2 << " ";  
}
```

會印出什麼？先自己列表格算一次。

``0 2 4 6`` —— ``i`` 從 0 開始、每次印 ``i*2``，跑到 ``i=3``（``3<4`` 成立）就結束，``i=4`` 不成立。

## 差一錯誤：`<=` 還是 `<` ？

```
for (int i = 1; i <= 5; i++) cout << i;    // 1 2 3 4 5 共 5 個  
for (int i = 1; i < 5; i++) cout << i;    // 1 2 3 4 共 4 個
```

這種「多印一個或少印一個」的錯誤叫**差一錯誤**（off-by-one error），是寫迴圈最常見的 bug，連老手都會中招。條件用 `<=` 還是 `<`，要看題目說的是「幾個」還是「到第幾個」。

## 常見情境：要印「n 個」東西

題目說「印出 n 個數字」，該怎麼寫迴圈條件？

```
for (i=1; i<=n; i++)
```

i 從 1 數到 n，共 n 個 ✓

```
for (i=0; i<n; i++)
```

i 從 0 數到 n-1，也是 n 個 ✓

兩種寫法都對，只是起點不同。**混著用才會出錯**——例如起點用 0、條件卻用 `i<=n`，就會多跑一次。

## 小測驗：這樣印幾個數字？

```
for (int i = 0; i <= 5; i++) {  
    cout << i << " ";  
}
```

**6 個**（`0 1 2 3 4 5`）。從 0 開始、條件是 ``<=5``，容易被誤以為是 5 個，其實是 6 個——這正是差一錯誤最容易發生的地方，起點跟終點都要仔細算。

## 迴圈也可以倒著數

```
for (int i = 5; i >= 1; i--) {  
    cout << i << " ";  
}
```

三元素完全一樣的邏輯，只是**初始值變大**、**條件變成** `>=`、**更新變成** `i--`。想倒數就把這三個地方一起改，只改其中一個會變成無窮迴圈或一次都不執行。

## 練習：倒數輸出

寫一個迴圈，把 `10 9 8 7 6` 印出來。

```
for (int i = 10; i >= 6; i--) {  
    cout << i << " ";  
}
```

起點 10、終點 6、每次減 1，條件用 `>=` 才會包含到 6 本身。

累加與累乘



## 累加求和 - 累加變數放錯位置會怎樣

```
for (int i = 1; i <= 5; i++) {  
    int sum = 0;        // ← 錯！每輪都歸零  
    sum += i;  
}  
cout << sum;           // 只會印出最後一輪的 5，不是 15
```

⚠ 如果把 `int sum = 0;` 寫進迴圈裡，每一輪都會被歸零，最後只會剩下最後一次加的那個數。累加變數一定要放在迴圈外面。

## 正確的寫法

```
// 計算 1 + 2 + 3 + ... + 100
int sum = 0;           // ← 一定要在迴圈外面
for (int i = 1; i <= 100; i++) {
    sum += i;          // sum = sum + i
}
cout << sum;          // 5050
```

## 累乘：起始值不一樣

```
int product = 1;           // ← 累乘從 1 開始，不是 0
for (int i = 1; i <= 5; i++) {
    product *= i;
}
cout << product;           // 120 (= 5!)
```

累加從 0 開始，因為「任何數 + 0 = 原本的數」。累乘要從 1 開始，因為「任何數 × 0 = 0」——如果從 0 開始，結果永遠是 0。

小測驗：這樣算出來是多少？

```
int sum = 0;
for (int i = 1; i <= 4; i++) {
    sum += i;
}
cout << sum;
```

**\*\*`10`\*\*——`1+2+3+4 = 10`。**

## 小測驗：找出錯誤

```
for (int i = 1; i <= 5; i++) {  
    int total = 0;  
    total += i;  
}  
cout << total;
```

這段程式碼有什麼問題？

``total`` 宣告在迴圈**裡面**，每輪都會被重設成 0；而且迴圈外的 ``cout`` 根本看不到迴圈裡宣告的 ``total``（會編譯錯誤）。應該把 ``int total = 0;`` 移到迴圈外面。

練習時間

## 第 6 題 累加與累乘

輸入  $n$  ( $1 \leq n \leq 10$ )，輸出總和與總乘積。

輸入

5

輸出

15

120

測資為 5，因此需輸出 "1 + 2 + 3 + 4 + 5" 及 "1 \* 2 \* 3 \* 4 \* 5"。

## 第 6 題 程式

```
int n;  
cin >> n;  
  
int sum = 0, product = 1;           // ← 起始值不一樣!  
for (int i = 1; i <= n; i++) {  
    sum += i;  
    product *= i;  
}  
cout << sum << endl;               // 第一行  
cout << product << endl;           // 第二行
```

⚠ 累加從 0 開始，累乘從 1 開始。輸出是兩行，不是一行兩個數字。



# 動手做

打開 [oj.itousouta.me](https://oj.itousouta.me) → 課程 Day3 → 第 6 題

寫到 AC 為止，再往下聽

break 與 continue

## break：整個迴圈直接結束

```
for (int i = 1; i <= 10; i++) {  
    if (i == 5) break;  
    cout << i << " ";    // 1 2 3 4  
}
```

一遇到 `break`，整個迴圈立刻停止，後面的輪次全部不執行，程式繼續往迴圈外面的下一行走。

## continue：跳過這一輪，繼續下一輪

```
for (int i = 1; i <= 10; i++) {  
    if (i % 2 == 0) continue;  
    cout << i << " ";    // 1 3 5 7 9  
}
```

遇到 `continue`，只跳過這一輪剩下的部分，直接跳到更新、檢查下一輪，迴圈本身不會結束。

## break 與 continue 比較

| 關鍵字                     | 效果             | 之後            |
|-------------------------|----------------|---------------|
| <code>`break`</code>    | 整個迴圈 <b>結束</b> | 跳到迴圈外面        |
| <code>`continue`</code> | 只跳過 <b>這一輪</b> | <b>繼續</b> 下一輪 |

``break`` 在「找到了就不用再找」的時候特別好用 —— 等一下質數判斷就會用到。

## 小測驗：這段會印出什麼？

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) continue;  
    if (i == 5) break;  
    cout << i << " ";  
}
```

**\*\*`1 2 4`\*\***——`i=3` 被跳過 (continue)，`i=5` 讓迴圈結束 (break，且結束前不會印出 5)。

## 應用：找出第一個能被 7 整除的數

```
int found = -1;
for (int i = 100; i <= 1000; i++) {
    if (i % 7 == 0) {
        found = i;
        break;           // 找到就停，不用繼續往後找
    }
}
cout << found << endl;    // 105
```

如果沒有 `break`，迴圈會繼續跑到 1000，`found` 最後會被後面找到的值蓋掉，答案就變成「最後一個」而不是「第一個」符合條件的數。

## 應用：跳過所有 3 的倍數

```
for (int i = 1; i <= 10; i++) {  
    if (i % 3 == 0) continue;  
    cout << i << " ";          // 1 2 4 5 7 8 10  
}
```

`continue` 適合「大部分都要處理，只有少數情況要跳過」的場景，比起把主要邏輯包在一個很大的 `if` 裡面，用 `continue` 先擋掉不要的情況，剩下的程式碼可以少縮排一層，讀起來更清楚。



# 質數判斷技巧

## 什麼是質數

- 質數是「只能被 1 和自己整除」的數 (2, 3, 5, 7, 11.....)
- 判斷方法：從 2 開始試除，只要找到一個能整除的數，就不是質數（用 `break`` 提早結束）
- 如果試到最後都找不到，才是質數

# 樸素解法

```
int n;  
cin >> n;  
bool isPrime = true;  
  
for (int i = 2; i < n; i++) {  
    if (n % i == 0) {  
        isPrime = false;  
        break;  
    }  
}
```

這樣寫對，但當  $n$  到  $10^6$ （一百萬）時，最壞情況要試將近一百萬次。還可以更快。

## 只要試到 $\sqrt{n}$

因數是成對出現的： $100 = 2 \times 50 = 4 \times 25 = 5 \times 20 = 10 \times 10 = 20 \times 5 \dots$

2×50

4×25

10×10

20×5

過了  $10 \times 10$  之後，後面全是前面配對的重複（ $20 \times 5$  其實就是  $5 \times 20$ ）。所以只要試到  $\sqrt{n}$  就夠了—— $n$  到  $10^6$  時，試除次數從一百萬次降到一千次。

練習時間

## 第 7 題 質數判斷

輸入  $n$  ( $2 \leq n \leq 10^6$ )，是質數輸出 ``Prime``，否則輸出 ``Not Prime``。

輸入

17

輸出

Prime

## 第 7 題 完整程式

```
int n;  
cin >> n;  
bool isPrime = true;  
  
for (int i = 2; i * i <= n; i++) {    // ← 只要試到  $\sqrt{n}$   
    if (n % i == 0) {  
        isPrime = false;  
        break;                      // 找到因數就可以停了  
    }  
}  
  
cout << (isPrime ? "Prime" : "Not Prime") << endl;
```

## 第 7 題 常見錯誤：n = 2 判斷錯誤

```
for (int i = 2; i * i <= n; i++) { ... } // n=2 時, i*i=4 > 2, 迴圈一次都不跑
```

`n = 2` 時迴圈一次都不會執行（`2\*2=4 > 2`），`isPrime` 保持初始值 `true`，剛好輸出 `Prime`——這其實是對的（2 是質數），但如果 `isPrime` 沒有正確初始化成 `true`，這種邊界情況就會出錯。永遠先想清楚最小的輸入會怎樣。



# 動手做

打開 [oj.itousouta.me](https://oj.itousouta.me) → 課程 Day3 → 第 7 題

寫到 AC 為止，再往下聽

while 與 do-while

## while 迴圈

```
// 先判斷，再執行
int n = 1;
while (n <= 5) {
    cout << n << " ";    // 1 2 3 4 5
    n++;                  // 別忘了更新，否則無窮迴圈！
}
```

- 適合「不知道要跑幾次，只知道結束條件」的情況
-  一定要有讓條件「變 false」的那一步，否則永遠跑不完

## 無窮迴圈：忘記更新會怎樣

```
int n = 1;
while (n <= 5) {
    cout << n << " ";
    // 忘記寫 n++;
}
```

`n` 永遠是 `1`，條件永遠成立，程式永遠不會停。在 itouOJ 上這種程式會一直跑到超過時間限制，判定為 **TLE**。

## do-while 迴圈

```
// 先執行，再判斷（至少執行一次）
int x = 10;
do {
    cout << "至少印一次" << endl;
    x++;
} while (x < 5);    // 條件一開始就 false，但仍印了一次
```

差別：`while` 先檢查條件，`do-while` 先做一次再檢查。

## while vs do-while : 同一個條件，結果不同

```
int x = 10;

// while : 條件一開始就不成立，一次都不會執行
while (x < 5) { cout << "A"; }

// do-while : 至少執行一次，才檢查條件
do { cout << "B"; } while (x < 5);
```

第一段完全不會印出東西；第二段會印一次 `B`，然後才發現條件不成立而停止。

## 三種迴圈怎麼選？

| 迴圈                      | 特性           | 適用時機        |
|-------------------------|--------------|-------------|
| <code>`for`</code>      | 次數明確         | 跑固定次數、走訪陣列  |
| <code>`while`</code>    | 先判斷，可能一次都不執行 | 次數未知、看條件    |
| <code>`do-while`</code> | 先執行一次，再判斷    | 至少要做一次（如選單） |

練習時間



## 第 8 題 數字翻轉

輸入正整數  $n$ ，反轉數字順序輸出（不含前導零）。

輸入

120

輸出

21

## 第 8 題 想法：一次拆一位數字

- `n % 10` 可以拿到  $n$  的個位數
- `n / 10` 可以把個位數砍掉（整數除法）
- 每拆一個位數，就把它接到答案的後面，直到  $n$  變成 0——不知道要拆幾次，正好用 `while`

## 第 8 題 程式

```
int n;  
cin >> n;  
int rev = 0;  
  
while (n > 0) {  
    rev = rev * 10 + n % 10;    // 取個位數，接到 rev 後面  
    n /= 10;                  // 砍掉個位數  
}  
cout << rev << endl;
```

## 第 8 題 $n = 120$ 的過程

| 輪次 | $n$ | $n \% 10$ | rev       |
|----|-----|-----------|-----------|
| 開始 | 120 | —         | 0         |
| 1  | 120 | 0         | 0         |
| 2  | 12  | 2         | 2         |
| 3  | 1   | 1         | <b>21</b> |

前導零自動就沒了 —— ``0`` 接在 ``0`` 後面還是 ``0``。不用特別處理 😊

# 動手做

打開 [oj.itousouta.me](https://oj.itousouta.me) → 課程 Day3 → 第 8 題

寫到 AC 為止，再往下聽



# 中場休息

15 分鐘後回來，我們進入「巢狀迴圈」

# 巢狀迴圈基礎

## 迴圈裡面再放迴圈

```
for (int i = 1; i <= 3; i++) {           // 外層: 列
    for (int j = 1; j <= 3; j++) {       // 內層: 行
        cout << i << ", " << j << " ";
    }
    cout << endl;                       // ← 位置很重要
}
```

```
1,1  1,2  1,3
2,1  2,2  2,3
3,1  3,2  3,3
```

外層跑一輪，內層就整圈跑完一遍。

`cout << endl;` 放在內層外面、外層裡面 —— 每印完一整列才換行。



## 內層不能跟外層共用同一個變數

```
for (int i = 1; i <= 3; i++) {  
    for (i = 1; i <= 3; i++) {    // ❌ 忘記寫 int, 沿用了外層的 i  
        cout << i << " ";  
    }  
}
```

內層迴圈結束時 `i` 已經被算到 `4` ——但這個 `i` 跟外層是**同一個變數**！外層迴圈拿到的 `i` 也變成 `4`，`4 <= 3` 不成立，外層迴圈直接被腰斬，只跑了 1 次。

外層跟內層是**各自獨立的計數器**，各管各的「跑幾次」，所以要給內層一個**新的變數名**（習慣用 `j`）：

```
for (int i = 1; i <= 3; i++) {  
    for (int j = 1; j <= 3; j++) {    // ✅ 內層用 j, 跟外層的 i 分開  
        cout << i << ", " << j << " ";  
    }  
}
```

## 執行次數：外層 × 內層

- 外層跑 3 次，**每一次**內層都要完整跑 3 次
- 總共執行內層的內容 `3 × 3 = 9` 次
- 這也是為什麼巢狀迴圈很容易讓程式變慢——次數是**相乘**的，不是相加

## 放錯位置的 endl 會怎樣

```
for (int i = 1; i <= 3; i++) {  
    for (int j = 1; j <= 3; j++) {  
        cout << i << j;  
        cout << endl;    // ← 放錯位置，變成內層的一部分  
    }  
}
```

這樣每印一個數字就換行一次，總共會換 9 次行，不是想要的「印完一整列才換行」。記住：`endl` 要跟內層迴圈的 `}` 同一層，不要寫進內層裡面。

## 小測驗：這段巢狀迴圈執行幾次？

```
for (int i = 1; i <= 4; i++) {  
    for (int j = 1; j <= 2; j++) {  
        cout << "*" ;  
    }  
}
```

**\*\*8 次\*\***——外層 4 次 × 內層 2 次 = 8 次，會印出 8 個星號。

## 練習：追蹤巢狀迴圈

```
for (int i = 1; i <= 2; i++) {  
    for (int j = 1; j <= 3; j++) {  
        cout << i * j << " ";  
    }  
    cout << endl;  
}
```

會印出什麼？先列表格追蹤外層、內層各自的值。

```
1 2 3  
2 4 6
```

外層 `i=1` 時，內層 `j` 跑 1~3，印出 `i\*j` = 1,2,3；外層 `i=2` 時同樣跑一輪，印出 2,4,6。

## 練手：完整九九乘法表

```
for (int i = 1; i <= 9; i++) {  
    for (int j = 1; j <= 9; j++) {  
        cout << i << "*" << j << "=" << i * j << "\t";  
    }  
    cout << endl;  
}
```

|       |       |       |     |
|-------|-------|-------|-----|
| 1*1=1 | 1*2=2 | 1*3=3 | ... |
| 2*1=2 | 2*2=4 | 2*3=6 | ... |

這是練習用的完整表格，用了巢狀迴圈。等一下 itouOJ 第 9 題只要印一個數的乘法表——會用到嗎？往下看就知道。

練習時間

## 第 9 題 九九乘法表

輸入  $n$  ( $1 \leq n \leq 9$ )，印出  $n$  的乘法表。

```
int n;  
cin >> n;  
for (int i = 1; i <= 9; i++) {  
    cout << n << " x " << i  
        << " = " << n * i << endl;  
}
```

$n = 3$  的完整輸出

```
3 x 1 = 3  
3 x 2 = 6  
3 x 3 = 9  
3 x 4 = 12  
3 x 5 = 15  
3 x 6 = 18  
3 x 7 = 21  
3 x 8 = 24  
3 x 9 = 27
```

⚠ 中間是小寫英文字母 ``x``，``x`` 和 ``=`` 前後都有一個空白。⚠ 這題只要一層迴圈就好，不是  $9 \times 9$ 。



## 第 9 題 常見錯誤：多寫了一層迴圈

```
for (int i = 1; i <= 9; i++) {  
    for (int j = 1; j <= 9; j++) {          // ← 不需要這層！  
        cout << n << " x " << j << " = " << n * j << endl;  
    }  
}
```

看到「乘法表」直覺想到剛剛的巢狀迴圈，但這題只印一個數 **n** 的乘法表，**n** 是輸入決定的固定值，不需要再用外層迴圈跑過 1~9。多寫的這層會讓同樣的內容重複印 9 遍——不是每個「表格」都需要巢狀迴圈。

# 動手做

打開 [oj.itousouta.me](https://oj.itousouta.me) → 課程 Day3 → 第 9 題

寫到 AC 為止，再往下聽

巢狀迴圈進階

# 內層條件依賴外層變數

```
int n;  
cin >> n;  
for (int i = 1; i <= n; i++) {  
    for (int j = 1; j <= i; j++) {  
        cout << "*";        // ← j 跑到 i, 不是固定的 n  
    }  
    cout << endl;  
}
```

```
*  
  
* *  
  
* * *  
  
* * * *
```

關鍵在內層條件是 `j <= i` 而不是 `j <= n` —— 每一行要印幾個，是跟著**行號**走的。這是「內層條件依賴外層變數」的第一個例子，第四天處理二維陣列時會大量用到。

## 逐步追蹤 n=4 的過程

| 外層 i | 內層條件    | 印出                  |
|------|---------|---------------------|
| i=1  | j 跑 1~1 | <code>`*`</code>    |
| i=2  | j 跑 1~2 | <code>`**`</code>   |
| i=3  | j 跑 1~3 | <code>`***`</code>  |
| i=4  | j 跑 1~4 | <code>`****`</code> |

外層的 ``i`` 除了控制跑幾列，同時也決定了內層要跑幾次。

練習時間

## 第 10 題 直角三角形

輸入  $n$  ( $1 \leq n \leq 20$ )，第  $i$  行印  $i$  個星號。

輸入

4

輸出

```
*  
**  
***  
****
```

跟剛剛教的「內層依賴外層」是同一件事，直接把那段程式拿來用即可。

# 動手做

打開 [oj.itousouta.me](https://oj.itousouta.me) → 課程 Day3 → 第 10 題

寫到 AC 為止，再往下聽



# 菱形技巧

## 找出規律

- 空白數：每往中間一行少 1
- 星號數：每往中間一行多 2，而且永遠是奇數
- 上半和下半剛好對稱，可以用同一組算式，只是方向相反

## 用 half 表示「半徑」

$n = 5$  時，`half = n / 2 = 2`（整數除法捨去小數）。把每一行用 `i` 從 0 數到 half 來表示：

| i      | 空白數 (half - i) | 星號數 (2i + 1) |
|--------|----------------|--------------|
| 0      | 2              | 1            |
| 1      | 1              | 3            |
| 2 (中間) | 0              | 5            |

上半是 `i` 從 0 數到 half；下半只是把同一組算式倒著再跑一次（`i` 從 half-1 數回 0）。

練習時間

## 第 11 題 菱形 (進階)

給定奇數  $n$ ，印出寬度  $n$  的星號菱形。先數清楚每行幾個空白、幾個星號。

```
  *  
 ***  
*****  
 ***  
  *
```

$n = 5$  :

| 行 | 空白 | 星號 |
|---|----|----|
| 1 | 2  | 1  |
| 2 | 1  | 3  |
| 3 | 0  | 5  |
| 4 | 1  | 3  |
| 5 | 2  | 1  |

## 第 11 題 拆成上下兩半

```
int n;  
cin >> n;  
int half = n / 2;           // n=5 → half=2  
  
for (int i = 0; i <= half; i++) {    // 上半 (含中間那行)  
    for (int j = 0; j < half - i; j++) cout << " ";  
    for (int j = 0; j < 2 * i + 1; j++) cout << "*";  
    cout << endl;  
}  
for (int i = half - 1; i >= 0; i--) { // 下半  
    for (int j = 0; j < half - i; j++) cout << " ";  
    for (int j = 0; j < 2 * i + 1; j++) cout << "*";  
    cout << endl;  
}
```

## 第 11 題 行尾不能有多餘空白

題目說行尾不要有多餘空白。空白只印在星號前面，印完星號就直接換行，不要在星號後面又補空白去對齊。

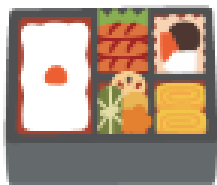
判題是逐字比對的，行尾多一個看不見的空白，也會被判定為 WA。

# 動手做

打開 [oj.itousouta.me](https://oj.itousouta.me) → 課程 Day3 → 第 11 題

寫到 AC 為止——今天 6 題全部完成！





# 短暫休息

10 分鐘後回來，做複習與小考

# 迴圈常見錯誤總複習

| 錯誤                    | 長什麼樣子                                     | 怎麼避免               |
|-----------------------|-------------------------------------------|--------------------|
| 差一錯誤                  | <code>&lt;=</code> 跟 <code>&lt;</code> 用混 | 先想清楚是「幾個」還是「到第幾個」  |
| 忘記更新                  | <code>while</code> 少了 <code>n++</code>    | 問自己：誰讓條件會不成立       |
| 累加變數放錯位置              | <code>sum=0</code> 寫在迴圈裡                  | 累加 / 累乘變數放迴圈外面     |
| <code>endl</code> 位置錯 | 放進內層迴圈裡                                   | 內層外面、外層裡面，才會一列換一次行 |

# 隨堂小測驗

# Q1

``for`` 迴圈的三元素，「更新」是在每輪的**開頭**還是**結尾**執行？

**\*\*結尾\*\***。順序是：檢查條件 → 執行內容 → 更新 → 再檢查條件。

## Q2

累加變數要放在迴圈的**裡面**還是**外面**？

**\*\*外面\*\***。放裡面的話每一輪都會被重新歸零。

## Q3

寫 `while` 迴圈之前，一定要先想清楚哪件事？

**\*\*是誰讓條件終究會不成立\*\***——答不出來就是無窮迴圈，會 TLE。

## Q4

巢狀迴圈裡，外層跑 4 次、內層跑 5 次，內層的內容總共會執行幾次？

**\*\*20 次\*\*** ( $4 \times 5$ ) 。

## Q5

``break`` 跟 ``continue`` 的差別是什麼？

**`break` 讓\*\*整個迴圈結束\*\*；`continue` 只跳過\*\*這一輪\*\*，迴圈繼續。**



## Q6

``for (int i = 1; i <= n; i++)`` 跟 ``for (int i = 0; i < n; i++)`` , 哪一個總共執行的次數比較多?

**\*\*一樣多\*\***, 都是  $n$  次, 只是起點跟終點不同。

## Q7

要把一段程式改成「倒著跑」，除了把 `i++` 改成 `i--`，還要改動哪兩個地方？

**\*\*初始值\*\***（改成起點在最大值）跟**\*\*條件\*\***（`<=` 改成 `>=`）。三個地方要一起改。

## Q8

累乘（例如算階乘）如果不小心把初始值設成 `int product = 0;`，最後答案會是多少？

**\*\*永遠是 `0`\*\*。**任何數乘以 0 都是 0，第一輪之後 product 就再也回不去了。

## Q9

用 `for (int i = 2; i * i <= n; i++)` 判斷質數，這樣寫比 `for (int i = 2; i < n; i++)` 快在  
哪裡？

因數是成對出現的，只要試到  $\sqrt{n}$  就能找到所有可能的因數， $n$  到  $10^6$  時試除次數從一百萬次降到約一千次。

## Q10

菱形圖案那一題，為什麼可以把上半跟下半拆成「同一組算式、方向相反」來寫？

因為菱形上下\*\*對稱\*\*：上半是空白遞減、星號遞增；下半剛好反過來，用同一個  $\text{half} - i$  跟  $2i + 1$  的算式，只是  $i$  的走向顛倒。

# 分組討論

## 討論①

如果程式交上去顯示 **TLE**（超過時間限制），你會優先懷疑哪裡出了問題？跟旁邊同學討論一分鐘。

## 討論②

跟旁邊同學互相解釋一次：為什麼九九乘法表的 ``cout << endl;`` 要放在內層迴圈外面、外層迴圈裡面。



## 討論③

三種迴圈 (for / while / do-while) 理論上可以互相改寫。跟旁邊同學討論：今天寫的 6 題裡，哪些題目改用別種迴圈也寫得出來？

今日重點回顧

## 回顧①：for 迴圈與累加累乘

- 三元素：初始值、條件、更新
- 卡住就手動列表格跑一遍
- 累加從 0、累乘從 1，變數要放在迴圈外面

## 回顧②：break/continue 與 while

- `break` 結束整個迴圈，`continue` 只跳過這一輪
- 質數判斷只要試到  $\sqrt{n}$ ，搭配 `break` 提早結束
- **while**：先判斷、可能一次都不執行；**do-while**：先做一次再判斷

## 回顧③：巢狀迴圈

- 執行次數是外層 × 內層
- 內層條件可以依賴外層變數 ( `j <= i` )
- 不是每個「表格」都需要巢狀迴圈——先看清楚題目要印幾層

## 回顧④：今天的節奏

|                     |        |
|---------------------|--------|
| 教了什麼                | 馬上練    |
| for 迴圈、累加累乘         | 第 6 題  |
| break/continue、質數技巧 | 第 6 題  |
| while/do-while      | 第 8 題  |
| 巢狀迴圈基礎              | 第 9 題  |
| 巢狀迴圈進階              | 第 10 題 |
| 菱形技巧                | 第 11 題 |

教一段、練一題，跟昨天一樣。

# 明天預告

明天我們要把一堆資料**一次裝起來** —— 陣列與字串。

今天學的迴圈，明天會拿來「走訪」一整排資料，兩個放在一起威力更大。

謝謝大家

明天見